Algorithm Lab
插件实现与发布学习手册

从银行家算法的运行过程出发，读懂 Python 记录器、浏览器播放器和编辑器插件如何协作。

学习版说明 2026年10月22日

先知道这个工具做了什么

你写的代码会先在本机真正执行。工具记录执行中的变量或你主动指定的画面，再把记录装入一个 HTML 文件，交给浏览器播放。因此，点击“上一步”是在回看记录，不是让 Python 程序倒着执行。

这份手册面向已经能编写基本 Python、正在学习操作系统算法的读者。读完后，你应能解释一帧画面的来源，修改一个可视化步骤，并把项目按清楚的来源和许可边界上传到 GitHub。

版本	用途	需要注意
0.1.1 本地安装版	你目前已经使用的版本	包含早期复用的上游 tracer 库
0.2.0 公开整理版	本次准备上传 GitHub 的源码	使用本项目新编写的记录层，导入 algorithm_lab

建议阅读顺序

先读第 2 至第 5 章，沿着一次按钮点击理解运行链路；再读第 7 章，把银行家算法和画面对应起来。准备动手改代码时读第 8 至第 11 章，准备发布时读第 12 和第 13 章。

本次没有覆盖或删除你的旧安装目录与学习项目。只阅读文档不需要更新插件；公开版示例和公开版运行器应成套使用。

02 一次点击如何变成动画

插件的核心作用是把当前文件路径和运行选项交给本地运行器。PyCharm 和 Visual Studio 使用不同的扩展接口，但之后会走到同一个 lab.py。



文件	负责什么
plugins/	读取当前文件，保存文档，启动运行器，提供开关
lab.py 和 worker.py	执行代码、收集记录、限制时间并处理错误
algorithm_lab.py 和 protocol.py	生成或还原可视化状态
viewer.html	将快照显示成代码高亮、数组、矩阵和日志

为什么使用子进程

算法可能运行很久，也可能抛出异常。把执行放到子进程，运行器就能收集输出、设置时限，并让编辑器保持响应。这只是执行隔离，不是权限受限的安全沙箱。

源代码主要留在本机处理。生成报告仍可能包含完整代码和绝对路径，所以公开分享报告前要检查其中内容。

03 PyCharm 插件入口

plugin.xml 注册菜单，Java 的 AnAction 处理点击事件。update() 决定按钮是否可用；actionPerformed() 负责真正启动。这条入口独立于原来的绿色运行按钮。

```
VirtualFile file = event.getData(CommonDataKeys.VIRTUAL_FILE);
var document = FileDocumentManager.getInstance().getDocument(file);
if (document != null)
    FileDocumentManager.getInstance().saveDocument(document);

List<String> args = List.of(python, labPath, file.getPath());
ProcessBuilder launcher = new ProcessBuilder(args);
launcher.environment().put("ALGORITHM_LAB_PYTHON", python);
launcher.start();
```

以上是删去界面处理后的关键顺序。实际代码先保存当前文档，再在后台线程启动进程。所选解释器通过环境变量继续传给 worker，避免“菜单选择了虚拟环境，真正执行却还是全局 Python”。

你遇到的路径错误

0.1.0 通过 Java 类的 CodeSource 取插件 JAR 路径。PyCharm 的插件类加载器可能不给这个位置，于是 getLocation() 返回空值，调用 toURI() 时失败。那时还没有启动 Python，重建 .venv 不会解决它。

```
var descriptor = PluginManagerCore.getPlugin(
    PluginId.getId("local.algorithm.lab"));
Path home = descriptor.getPluginPath().resolve("runtime");
```

0.1.1 及公开版通过 IDE 的插件描述信息定位安装目录，再检查 runtime/lab.py 是否存在。这个修复说明：插件加载成功、解释器正确、算法运行成功，是三个不同的验证环节。

对应源码：plugins/pycharm/plugin.xml、AlgorithmAction.java、RuntimeLocator.java。

04 普通 Python 怎样逐行记录

普通代码没有“请画一个矩阵”的语句。worker.py 使用 sys.settrace() 接收当前文件的执行事件，记录行号、函数名和局部变量。它不需要理解算法的语义。

```
def trace(frame, event, arg):
    if frame.f_code.co_filename != target:
        return trace
    if event == "line":
        snapshot(frame, frame.f_lineno, "before")
    elif event == "return":
        snapshot(frame, frame.f_lineno, "return")
    return trace

sys.settrace(trace)
runpy.run_path(target, run_name="__main__")
```

高亮行表示即将执行

line 事件出现在该行执行之前。例如 total += number 高亮时，右侧 total 仍是相加之前的值。下一次事件到来，才能看到相加后的结果。工具另外保留返回与异常事件，因此最后一步不一定是“执行前”。

执行位置	高亮前看到的变量
total = 0	total 可能还没有创建
第一次 total += number	total = 0，number = 5
第二次 total += number	total = 5，number = 2
print(total)	total = 15

记录器会把能处理的值转换成普通数据，并限制深度、列表长度和步数；它不会任意调用用户对象的 repr()。当前仅追踪这个文件和当前线程，也没有完整的调用栈界面。

官方技术说明见参考资料中的 Python sys.settrace。以上机制描述对应当前项目的实现，不代表所有调试器都以相同方式展示状态。

05 怎样指定要画出来的状态

银行家算法需要表达“这个数组是当前可用资源” “这一行是正在检查的进程” 。公开版通过 algorithm_lab.py 的小型接口，让代码明确指定这些含义。

```
from algorithm_lab import ArrayView, MatrixView, Journal, step

work = [3, 3, 2]
view = ArrayView("Work", work)
log = Journal("Process")
step()

work[0] += 2
view.update(0, work[0])
log.write("P1 completed and released resource A")
step()
```

ArrayView 保存一维数组，MatrixView 保存二维矩阵，Journal 保存说明文字。update() 改变展示的数据，step() 把当前画面复制成一帧，并自动获取调用处行号。

为什么修改 work 后还要 update

算法变量和展示视图是分开的。视图创建时复制输入，避免后续修改把历史画面一起改掉。因此 work[0] += 2 改的是算法数据，view.update(...) 才明确更新画面。这个设计让你决定哪些细节值得展示。

```
_frames.append({
    "line": line,
    "panels": deepcopy(list(_views.values()))
})
```

深复制是回看的基础。若历史帧只是指向同一份列表，程序结束后每一帧可能都变成最终值。

迁移提醒：0.1.1 示例使用 algorithm_visualizer 和 Tracer.delay；公开版使用 algorithm_lab 和 step。完整对应表在 docs/MIGRATION.md。

06 记录如何变成统一的画面数据

浏览器不需要知道源代码是 Python 还是 C。它只需要一组带行号的画面快照。公开版 Python 记录层直接导出 frames；C/C++ 接口先写命令，再由 protocol.py 还原成同样的 panels。

```
{
  "line": 36,
  "panels": [{
    "key": "view-3",
    "kind": "Array1DTracer",
    "title": "Work",
    "data": [5, 3, 2],
    "selected": [],
    "patched": ["0"]
  }]
}
```

kind 是本地播放器识别面板类型的标签，并不表示公开包导入了同名的上游类。data 决定数值，selected 标记正在检查的元素，patched 标记代码主动更新的元素。

命令记录与状态快照

形式	一个操作的例子	怎样回看
命令	把 Work 下标 0 改为 5	依序应用命令，再保存一帧
快照	直接保存 Work = [5,3,2]	直接读取第 i 帧的数据

早期本地版复用上游 tracer 的命令格式。公开版新 Python 记录层改为直接保存状态；C/C++ 仍使用本项目自己的命令写入接口。这两条路径最终都交给同一个播放器。

这种回放保存的是观察结果。回退到前一帧时，并不会重新分配真实操作系统资源，也不会撤销用户程序的文件写入。

07 把银行家算法与画面对上

安全性检查先计算 Need = Max − Allocation，再令 Work = Available。寻找满足每一种资源都有 Need[i][j] <= Work[j] 的未完成进程；假设它完成后，把它已占有的资源加回 Work。

检查进程	Need	检查前 Work	结果与检查后 Work
P0	[7,4,3]	[3,3,2]	跳过，资源不足
P1	[1,2,2]	[3,3,2]	完成 → [5,3,2]
P2	[6,0,0]	[5,3,2]	跳过，A 不足
P3	[0,1,1]	[5,3,2]	完成 → [7,4,3]
P4	[4,3,1]	[7,4,3]	完成 → [7,4,5]
P0	[7,4,3]	[7,4,5]	完成 → [7,5,5]
P2	[6,0,0]	[7,5,5]	完成 → [10,5,7]

于是安全序列为 P1 → P3 → P4 → P0 → P2。动画中的“完成”是安全性算法的模拟，不是实际终止系统进程。Work 是检查过程中的工作向量，不需要把原始 Available 变量改成最终值。

为什么示例是十四步

当前示例记录 1 帧初始状态、7 帧检查、5 帧成功完成，以及 1 帧最终结论，共 14 帧。逐行模式会记录循环和判断等更多执行事件，所以历史报告出现 318 步并不代表算法结果不同。

把 Available 改为 [0,0,0] 后，没有进程能首先完成，找不到完整安全序列。结论是“不安全”；不能据此说它必然已经死锁。示例尚未实现新资源请求的试分配、回滚等完整流程。

08 播放器怎样播放与回退

viewer.html 把快照数组和界面控件放在一个文件里。index 表示当前播放到哪一帧，render() 把这一帧的面板和行号显示出来。

```
function seek(n) {
  index = Math.max(0, Math.min(frames.length - 1, n));
  render();
}

function next() { seek(index + 1); }
function previous() { seek(index - 1); }

timer = setInterval(() => seek(index + 1), interval);
```

这里是去掉按钮细节的结构示意。播放只是定时增加 index；暂停清除定时器；回退减少 index。拖动进度条直接选择一帧，不需要让算法重新执行。

代码高亮和数值标记

每行源代码都有独立的行号元素。frame.line 指向当前行，矩阵数据生成表格单元格。正在检查的项、主动更新的项、与前一帧数值不同的项分别使用不同样式。窄窗口下，代码区通过元素的位置差计算滚动距离，避免高亮行跑到可视区域外。

把代码作为文本显示

用户代码可能包含 </script> 等字符串。生成 HTML 时，运行器把数据中的小于号转义；浏览器显示标题、日志和代码时使用 textContent，避免把这些文本当作 HTML 标签执行。测试覆盖了嵌入脚本结束标签的情况。

这不等于整个运行环境是安全沙箱。用户程序仍然在本机执行，播放器只负责展示结果。

09 C 与 C++ 怎样接入

公开版不再使用上游 C++ tracer 或 JSON 库。include/algorithm_lab_c.h 使用标准输入输出函数写 JSON，C 和 C++ 示例共用这个头文件。

```
#include "algorithm_lab_c.h"

int main(void) {
    int work[3] = {3, 3, 2};
    lab_create("work", "Array1DTracer", "Work");
    lab_array1d("work", work, 3);
    lab_step(__LINE__);
    work[0] += 2;
    lab_array1d("work", work, 3);
    lab_step(__LINE__);
    return 0;
}
```

lab_create 创建面板，lab_array1d 记录数组，lab_step(__LINE__) 标记一步及源代码行号。二维矩阵和日志也有对应接口。JSON 字符串中的引号、反斜杠和控制字符会进行转义。

编译和运行链路

lab.py 查找 MSVC，设置头文件搜索目录，把当前源文件编译为临时可执行文件。运行时通过 ALGORITHM_LAB_TRACE 指定记录文件，结束后读取命令并生成 HTML。编译错误和非零退出码会进入报告。

这是单源文件教学入口。它不复用复杂 .vcxproj 的链接配置，不自动读取任意 C/C++ 变量。当前头文件的状态按单个编译单元设计，多文件或多线程需要额外设计，不能简单把接口加到所有文件就认为已支持。

插件只调用本机已安装的编译器；公开包不附带 Visual Studio SDK、MSVC 或 Python。

10 从遇到过的问题学习排查

现象	应检查什么	对应处理
普通运行输出了结果，却没有动画	是否点击原来的绿色运行按钮	从 Algorithm Lab 菜单启动
看到变量列表，没看到资源矩阵	报告标题是否为 Python 逐行	对记录过的算法选可视化当前文件
CodeSource 位置为空	插件定位安装目录失败	更新到修复版本并重启 IDE
导入处画红线	仅静态检查，还是实际运行报错	先通过插件入口实际验证
导入旧库时报错	是否混用了 0.1.1 的示例	按迁移表改为 algorithm_lab
项目需要的第三方包找不到	worker 是否用项目的解释器	在插件设置中选择项目 .venv

把问题分成四层

第一层是编辑器有没有加载插件；第二层是插件能不能启动 Python；第三层是你的程序能不能执行；第四层是记录能不能正确显示。一个步骤通过，不能替代其余步骤的验证。

例如，截图中普通运行输出 15 只说明项目解释器能运行那段代码；后面的 Java 弹窗属于第二层。另一次银行家报告的算法结果正确，但选了逐行模式，属于展示入口的差别。

不用一次改动很多环境设置

先保存错误文字、报告模式和一次最小代码，再定位具体层级。修改一个因素后重新验证。这样比删除虚拟环境或重新安装全部依赖，更容易知道究竟是什么解决了问题。

11 怎样验证和动手修改

公开版在本机完成 13 项自动检查，包括 Python 与 C/C++ 的安全和不安全输入、快照独立性、行号、异常、超时、HTML 转义和模式选择。测试证明这些具体行为，不代表任意程序或所有 IDE 版本都已兼容。

```
python -m unittest discover -s tests -v
python lab.py examples/banker.py
python lab.py examples/banker.py --mode lines
```

练习	先改哪里	应该看到什么
改变安全性	available 改成全零	最终日志为不安全，序列为空
减少动画步骤	删除一次 step()	该检查不再单独停留，最终计算不变
增加说明	在完成进程后 log.write	日志增加一句，算法值不变
观察函数内变量	用普通模式运行带函数的代码	逐行模式显示当前函数局部变量
改变界面颜色	viewer.html 的 CSS	重新生成报告后样式变化

从小修改开始理解依赖

先只改输入，再改记录语句，最后改播放器。修改仓库源码后，需要重新构建并安装插件，才能更新已安装的运行器副本。生成报告和缓存应留在本地，避免把私人代码意外提交。

两个公开版插件已经用本机 IDE 编译组件构建，打包运行时也要单独检查。用户实际确认过的是 0.1.1 的 PyCharm 入口；公开整理版 0.2.0 仍需要在 IDE 中安装后确认加载。Visual Studio 的实际加载状态也不能仅凭编译成功判断。

12 公开发布时怎样处理来源与许可

可以把这个学习项目整理到自己的 GitHub。关键是对你实际分发的文件逐一确认来源和许可，而不是看到某个相关仓库标了 MIT，就把所有关联仓库都当作 MIT。

组件或文件	本次决定	原因
原网页源码快照	公开包不包含	虽然有 MIT，也没有必要复制整套网页
上游两个 tracer 库	不包含	对应快照没有找到明确许可证
旧版 ZIP 和 VSIX	不包含	它们绑定了上述库，换个外壳仍是在分发
新记录层与 C 接口	包含源码	为本项目编写，公开版不依赖上游 tracer
MIT 与来源说明	放在根目录	表明本包许可范围与设计来源

GitHub 官方说明，没有许可证时默认版权规则适用；公开可见、可以查看或 fork，不等于任意再分发。MIT 则允许在符合条件时使用和分发，其中包括保留版权与许可声明。

不冒充上游产品

README 保留 Algorithm Visualizer 的设计致谢，说明项目没有官方隶属关系，并如实标注 AI 辅助开发。MIT 只覆盖本包中有权授权的内容，不会替外部代码自动补上授权。项目版权行暂用 Algorithm Lab contributors，你可以按实际贡献情况补充自己的账号。

这是一份按已确认信息作出的分发整理，不是法院的权利认定，也不能保证所有可能争议都不存在。若未来加入新的库或发布商业产品，应针对对应版本再次核查。

13 把文件上传成一个能阅读的仓库

建议在自己的账号下新建独立仓库，例如 algorithm-lab-local。先解压本次上传包，再把里面的源码文件夹内容放到仓库根目录；只上传 ZIP 本身，GitHub 不会自动把它展开成源码。

```
README.md          # overview and quick start
LICENSE            # permission terms for this package
THIRD_PARTY.md     # provenance and exclusions
lab.py
worker.py
algorithm_lab.py
viewer.html
examples/
include/
plugins/
tests/
docs/
.github/workflows/tests.yml
.gitignore
```

上传后，根目录应直接显示 README.md 和 lab.py。保留 .github、.gitignore 与 LICENSE。若加入已有仓库，使用独立子目录并保留该目录的许可，不覆盖现有文件。

不要把本地研究目录整个上传

本次 ZIP 已排除 vendor、source-reference、旧安装包、.build、.scratch、虚拟环境、IDE 设置和带个人路径的报告。.gitignore 只影响 Git 的忽略规则，不能阻止你在网页上手动上传隐私文件。

后续从公开版源码构建的插件，可以在本机 IDE 验证后作为 Release 附件发布。仓库提供的 CI 主要运行普通 Python 测试；没有实际运行的远程工作流，不能写成已经获得绿色通过。本次没有创建远程仓库或推送代码。

下一步可以做什么

先完成一个输入修改和一个画面修改，并记录为什么这样改。之后再考虑自动选择项目解释器、增加调用栈视图，或接入真实调试器。实时调试需要在程序运行中控制暂停与继续，这需要新的执行架构，不能只调整播放速度。

14 参考资料与源码入口

学习时优先对照公开包中的实际源码。外部链接用于理解接口和许可背景；网页版本可能更新，本文核查日期为 2026年10月22日。

Python 官方文档 sys.settrace 执行事件

对应第 4 章。注意 line 事件的执行前语义以及追踪的线程范围。

Python 官方文档 copy 与 deepcopy

对应第 5 和第 6 章。重点理解变量赋值保存引用与历史快照保存内容的差别。

JetBrains 官方文档 Action System

对应第 3 章。区分动作实现、注册位置和按钮可用状态。

Algorithm Visualizer 设计来源

最初选择的参考项目；公开包保留致谢，不包含其完整代码副本。

GitHub 官方文档 Licensing a repository

对应第 12 章。公开托管与开源许可不是同一个判断。

Open Source Initiative MIT License

对应本包 LICENSE。保留版权和许可声明，且不能据此替外部组件决定授权。

随项目提供的阅读材料

文件	用途
README.md	启动、构建、能力范围
docs/MIGRATION.md	0.1.1 到 0.2.0 的接口变化
docs/PUBLISHING.md	解压后上传的顺序和目录检查
THIRD_PARTY.md	组件来源、核查提交与排除范围